

Unlocking Untapped Potential: A Technical Assessment of Software-Driven Performance on Legacy Ubiquiti airMAX AC Hardware

This report presents an exhaustive, evidence-based investigation into the potential for achieving material performance improvements—specifically in capacity, spectral efficiency, latency, fairness, and stability—on legacy Ubiquiti airMAX AC hardware. The analysis focuses exclusively on higher-layer software, firmware, and algorithmic optimizations, strictly respecting the hardware boundaries of the Qualcomm/Atheros QCA988x/QCA9882-class radios. It explicitly excludes any attempt to emulate Wi-Fi 6/7 or introduce unsupported physical-layer capabilities. The research synthesizes findings from primary sources including peer-reviewed papers, Linux kernel mailing lists, OpenWrt documentation, FCC filings, GPL releases, and real-world test reports to construct a detailed map of opportunities and critical blockers. The final output provides a ranked list of transferable techniques, identifies hard constraints imposed by the hardware/firmware paradigm, and outlines the most critical questions that must be answered through physical experimentation.

Hardware Architecture and Firmware Offloading Constraints

The potential for software-driven improvement on Ubiquiti's airMAX AC hardware is fundamentally defined by its underlying architecture, which centers on the Qualcomm Atheros QCA988x family of chips [40](#) [41](#). These chips are capable IEEE 802.11ac radios, but their integration into a consumer-grade WISP product is mediated by a complex software stack involving the `ath10k` Linux wireless driver, a highly offloaded radio firmware, and Ubiquiti's proprietary `airOS`. Understanding the constraints imposed by this stack is paramount to assessing the feasibility of any optimization effort. A major architectural decision is the extensive use of firmware offloading, where the host CPU delegates much of the Medium Access Control (MAC) layer processing to the radio's

onboard processor [26](#) [41](#). While this design choice was made to maximize raw throughput by reducing host CPU overhead, it creates significant barriers for higher-layer optimization by creating a semi-closed system with limited visibility and control [26](#).

One of the most severe limitations stemming from this architecture is the lack of reliable transmit completion telemetry. The `ath10k` driver, while functional, has a well-documented flaw on QCA988x hardware: it fails to report the actual transmission rate used for a given packet in its completion events [40](#). As a result, user-space tools attempting to monitor transmission rates will almost universally report an incorrect value, often defaulting to 6 Mbps regardless of the actual MCS/NSS combination being used [40](#). The only accurate way to obtain this information is by parsing specialized debug files located at `/sys/kernel/debug/ieee80211/phyX/ath10k/fw_stats` [40](#). This telemetry gap is a critical blocker for nearly all modern adaptive rate control algorithms, such as Minstrel HT, which rely on a continuous feedback loop of packet success and failure rates at specific modulation and coding schemes to dynamically adjust the link's performance [95](#). Without knowing what rate was actually transmitted, the software cannot learn from past transmissions and adapt to changing channel conditions. This single constraint invalidates many advanced rate-adaptation schemes and represents a fundamental limitation of the platform.

Beyond the driver-level telemetry issue, Ubiquiti's stock firmware, `airOS`, has demonstrated inconsistent behavior that further complicates network-wide optimization. Users have reported that signal level readings can fluctuate seemingly without cause due to Automatic Gain Control (AGC) compensation during RSSI calculation [24](#) [53](#) [55](#). Furthermore, metrics like Channel Capacity Quotient (CCQ) have been observed to drop unexpectedly, and some users have experienced complete loss of CLI responsiveness under load [53](#) [68](#). These instabilities and unreliable telemetry make it difficult for an external controller to build a coherent model of the network's state. An effective cross-layer optimization strategy requires consistent, accurate data to make intelligent decisions, and the current firmware environment appears to provide neither. This suggests that before any sophisticated algorithm can be deployed, stabilizing the base operating system is a prerequisite.

The history of `ath10k` development reveals other instances where ambitious MAC-layer projects were either abandoned or remain experimental due to firmware offloading challenges. For example, a series of patches proposed around 2018 aimed to add a new Transmit Queue (TXQ) scheduling API to the core Linux `mac80211` subsystem [26](#). The goal was to enable more sophisticated queue management algorithms directly within the MAC layer. However, developers noted that implementing such features was fraught with

difficulty because they required the host CPU to handle all MAC processing, which ran counter to the performance goals of firmware offloading [26](#). This indicates that even when the kernel community develops powerful new features, their practical application to `ath10k`-based hardware may be hindered by fundamental architectural trade-offs. Similarly, early work on airtime fairness patches for the older `ath9k` driver built upon the existing `fq_codel` and ATF (Advanced Traffic Filter) infrastructure, showing a mature ecosystem for that driver/driver-architecture combination [95](#). The parallel effort for `ath10k` faced unique challenges, highlighting the chipset-specific nature of these problems [25](#). This historical context serves as a cautionary tale, suggesting that even theoretically sound techniques may prove unstable or incomplete in practice on QCA988x hardware.

In summary, the hardware architecture of the target Ubiquiti devices, while based on capable QCA988x/QCA9882-class radios, imposes severe constraints on software optimization. The combination of firmware offloading, a critical lack of transmit rate telemetry from the firmware, and documented inconsistencies in the stock `airOS` telemetry creates a challenging environment. Any attempt to implement novel MAC-layer optimizations must contend with a "black box" view of the radio's operations, making adaptive and predictive algorithms exceptionally difficult to deploy. The most promising avenues for improvement lie outside the direct control of the radio's MAC scheduler, focusing instead on host-side traffic shaping and queuing discipline management.

The airMAX TDMA Protocol: An Investigation into Its Capabilities and Limitations

Ubiquiti's airMAX AC technology is built upon a proprietary Time Division Multiple Access (TDMA) protocol designed specifically for outdoor point-to-multipoint (PtMP) environments [15](#) [23](#). This protocol is Ubiquiti's primary mechanism for mitigating the hidden node problem, a common issue in WISP deployments where stations cannot hear each other but are within range of the central AP [20](#) [23](#). By assigning dedicated time slots to each subscriber station (CPE), the airMAX TDMA protocol ensures that only one station transmits at a time, thereby eliminating collisions and maximizing airtime efficiency [20](#) [70](#) [101](#). This approach is credited with providing significant performance improvements in latency, scalability, and throughput compared to traditional CSMA/CA-based Wi-Fi systems [15](#) [16](#). The system keeps track of active stations and allocates airtime accordingly, with idle stations potentially having no dedicated slot [117](#). Ubiquiti holds

patents related to an "Adaptive Synchronous Protocol for Minimizing Latency in TDD Systems," which further underscores their investment in this technology [84](#).

Despite its effectiveness, the airMAX TDMA implementation presents significant limitations for the purposes of third-party optimization. The entire system operates as a closed, black-box component deep within the airOS firmware. There is no public documentation detailing the internal logic of the scheduler, the criteria used for slot allocation, or the parameters governing its adaptive behavior. While users can access certain settings through the web GUI, such as the "TDMA Filter" setting which, if set to zero, causes M-series clients to show AirMax as disabled, the core algorithm remains opaque [112](#). This lack of transparency makes it impossible to perform cross-layer optimization or to implement novel scheduling strategies without risking instability or regression. An external controller cannot easily interface with or augment the proprietary scheduler; it can only enable or disable the entire TDMA mode, reverting the device to a standard 802.11ac mode, which is not the research goal [52](#).

Furthermore, the performance of the airMAX protocol, while generally robust, is subject to firmware-related issues that can degrade its effectiveness. Different versions of airOS have been reported to alter performance characteristics significantly. For instance, firmware version 8.7.0 reportedly introduced connection issues, with users finding that downgrading to 8.6.2 resolved the problems instantly, suggesting a firmware-induced regression [50](#). Other updates have brought PTP performance improvements and reduced CPU load on higher channel widths, indicating that the scheduler's behavior is actively being modified by Ubiquiti [57](#). However, these changes are not always positive. Some users have found that newer firmwares break compatibility with management systems like AirControl2 [54](#), and others have encountered bugs that cause devices to become unresponsive after an upgrade [71](#). This volatility suggests that relying on the proprietary scheduler is akin to building on shifting sand, making it a poor foundation for a stable, long-term optimization project like FuturaMAX.

The reliance on GPS synchronization for TDMA coordination, particularly on models like the LiteAP GPS, adds another layer of complexity and dependency [23](#). While GPS provides a highly accurate timing reference essential for precise slot allocation across a sector, it also introduces a dependency on receiving a valid GPS signal, which can be problematic in urban canyons or under heavy foliage. The firmware itself includes logic to manage GPS time sync, with bug fixes appearing in recent releases to improve this functionality [51](#). However, the interaction between the GPS timing module, the main scheduler, and any potential external controller is entirely undocumented. Attempting to run a secondary, unsynchronized TDMA system would likely lead to catastrophic

interference. Therefore, any external optimization must either operate in concert with the existing GPS-synchronized system or risk rendering it non-functional.

In conclusion, while Ubiquiti's proprietary airMAX TDMA protocol is a powerful tool for managing PtMP networks and effectively solves the hidden-node problem, its closed, opaque nature and susceptibility to firmware-induced regressions present formidable obstacles to third-party enhancement. The lack of public APIs, control interfaces, or telemetry for the scheduler prevents any form of meaningful cross-layer optimization. Future efforts to improve upon this system would require either a breakthrough in reverse-engineering the firmware's internal workings or a significant shift in Ubiquiti's open-source strategy to expose the necessary hooks for external control. For now, the airMAX TDMA engine must be treated as an immutable block in the overall system architecture.

Advanced MAC Scheduling and Hidden-Node Mitigation on Commodity Hardware

The theoretical possibility of implementing advanced MAC scheduling and TDMA on commodity Wi-Fi hardware like the QCA988x has been explored extensively in academic and open-source communities. These efforts aim to move beyond the basic collision avoidance of CSMA/CA or the closed nature of proprietary protocols like airMAX TDMA. Projects such as hMAC, Det-WiFi, and SoftMAC, along with foundational work from the MadWifi project on FreeBSD, demonstrate that it is feasible to build custom TDMA and scheduled MAC systems on top of standard 802.11 hardware . These systems typically involve a central controller that polls stations and schedules their transmissions, a concept that aligns with the principles of coordinated TDMA (Co-TDMA) discussed in numerous research papers [88](#) [89](#) [108](#). Such schemes have been shown to effectively reduce latency and improve medium utilization in multi-access scenarios [108](#).

However, the practical implementation of these ideas on the target Ubiquiti hardware faces a monumental challenge: firmware offloading. The `ath10k` driver, while providing the necessary low-level interface, is designed to delegate MAC processing to the QCA988x's onboard firmware [41](#) . This means that the host CPU, running an external scheduling controller, has limited ability to directly command the radio to transmit at a specific time or to inspect the state of the radio's internal queues. As previously noted, attempts to develop more sophisticated TXQ scheduling APIs within the Linux kernel

itself were hampered by this very issue, as they required a fully host-computed MAC layer to function, sacrificing the throughput benefits of offloading [26](#). Therefore, any external TDMA scheduler would need to communicate its schedule to the radio's firmware, hoping that the firmware can interpret and execute these commands correctly.

The exact mechanism for such communication is unknown. Ubiquiti's REST API allows for some automation of device configuration, such as activating SSH on a batch of devices, but it is unclear if it provides low-level primitives for manipulating the TDMA scheduler's internal state [97](#). Without such an interface, an external controller would be limited to enabling/disabling the global TDMA mode and perhaps adjusting high-level parameters, leaving the intricate details of scheduling up to the black-box `airOS` firmware. This renders the external controller largely ineffective. One could envision a hybrid model where an external controller manages a coarse-grained schedule, while the firmware handles fine-grained arbitration within those larger time blocks, but the feasibility of this approach is purely speculative and remains UNKNOWN due to a complete lack of documentation on the firmware's programmability.

Another avenue for hidden-node mitigation involves the use of RTS/CTS (Request to Send/Clear to Send). The 802.11 standard specifies RTS/CTS as a mechanism to address the hidden node problem by reserving the medium before a data transmission occurs [67](#). However, its effectiveness is highly dependent on proper configuration and the specific network topology. In a typical WISP scenario with a central AP and many distributed CPEs, using RTS/CTS can be inefficient, as the handshake overhead can consume significant airtime, especially for large data frames. Modern Wi-Fi drivers and protocols often employ dynamic RTS/CTS thresholds, where the RTS/CTS handshake is only used for frames larger than a certain size. The availability and configurability of these parameters on the `ath10k` driver for QCA988x are not well-documented in the provided sources, but it is a feature present in many `mac80211`-based drivers. Optimizing these thresholds could offer some improvement in congested hidden-node scenarios, but it is unlikely to provide the dramatic gains seen with a true scheduled MAC like `airMAX` TDMA.

More advanced concepts like spatial reuse, enabled by features like ANI (Adaptive Noise Immunity), are also part of the broader effort to mitigate interference. ANI allows a receiver to distinguish between noise and weak signals from distant access points, potentially allowing it to decode a desired signal even when the CCA (Clear Channel Assessment) would otherwise indicate the channel is busy. However, the extent to which these features are exposed and controllable via software on the `ath10k` driver is again, largely unknown. The QCA988x does support spectral scan features, which can be used for interference classification, but using this data for real-time, proactive interference

mitigation requires a sophisticated algorithm and a fast reaction time that may be difficult to achieve with a host-based controller interacting with a firmware-managed radio [48](#) [66](#). The firmware may already have its own mechanisms for handling interference, and an external controller might conflict with these native behaviors rather than augment them.

In essence, while the academic and open-source communities have successfully built advanced MAC schedulers on commodity hardware, their direct applicability to the target Ubiquiti devices is severely curtailed by the opaque, firmware-offloaded architecture. The lack of a clear, documented method for an external controller to exert fine-grained control over the radio's transmission timing and scheduling makes the implementation of a superior TDMA system a significant, currently insurmountable challenge. The most realistic hope lies in finding a way to interface with or augment the existing firmware scheduler, a task that borders on reverse-engineering and carries considerable risk of instability.

Rate Adaptation, Aggregation, and Telemetry Gaps on QCA988x

The performance of any wireless link is critically dependent on its ability to adapt to fluctuating channel conditions. This is achieved through dynamic rate adaptation, where the transmitter adjusts its Modulation and Coding Scheme (MCS) and Number of Spatial Streams (NSS) based on link quality feedback. State-of-the-art rate control algorithms like Minstrel HT and its derivatives (e.g., IteRate) are designed to maximize goodput by intelligently selecting the highest possible rate that maintains a low packet error rate [95](#). These algorithms are considered a cornerstone of modern Wi-Fi performance and are widely implemented in open-source drivers like `ath9k` [95](#). However, their effectiveness is predicated on a crucial piece of telemetry: the ability to determine the actual transmission rate used for every packet sent.

As established, the `ath10k` driver for QCA988x hardware suffers from a critical deficiency in this regard: it does not reliably report the transmit rate in tx completion events [40](#). This single limitation acts as a hard blocker for implementing any advanced adaptive rate control algorithm on this platform. Without knowing the rate at which a frame was transmitted, the software cannot correlate packet successes or failures with a specific MCS/NSS pair. Consequently, it cannot build a statistical model of link

performance or adjust its rate selection strategy over time. Attempts to use these algorithms on QCA988x hardware would be akin to trying to drive a car with no speedometer; the system could select a rate, but it would have no way of knowing if that rate was optimal, too aggressive, or too conservative. This telemetry gap is a definitive REFUTED finding for any technique requiring this level of granular feedback. The only workaround is to parse kernel debugfs files, a process that is too slow and resource-intensive for real-time rate adaptation [40](#).

This telemetry gap has profound implications for other aspects of link control as well. Adaptive aggregation, which involves bundling multiple frames into a single A-MPDU (Aggregated MPDU) to improve efficiency, is another area where feedback is key. The optimal size of an A-MPDU is a trade-off between efficiency (more frames per transmission) and resilience to errors (a single corrupted frame corrupts the entire A-MPDU). Algorithms that can dynamically adjust A-MPDU length based on packet error rates are beneficial. However, without reliable feedback on transmission success and the associated rate, tuning these parameters becomes guesswork. Controls for A-MPDU and A-MSDU (Aggregated MSDU) generation are often managed by the firmware in offloaded drivers, and their availability and configurability via standard Linux tools like `iw` are typically limited. While some research mentions EDRA (Enhanced Data Rate Adaptation) as a technique, its practical implementation is contingent on the same telemetry that is unavailable on QCA988x.

The instability of rate masks, mentioned anecdotally by users, may be a symptom of this deeper problem. If the software is unable to accurately assess link quality, it may resort to conservative or oscillating rate choices, leading to suboptimal performance. The existence of alternative firmware from Candela Technologies (`ath10k-firmware-qca988x-ct`) suggests that there is interest in improving the base firmware's behavior, as it enables features like IBSS (Independent Basic Service Set) that are disabled in the official firmware [38](#) [39](#). However, there is no indication in the provided sources that these alternative firmwares solve the fundamental telemetry problem for rate reporting. They may improve stability or enable new features, but they do not appear to provide the necessary hooks for advanced, software-driven rate adaptation.

In conclusion, the inability to obtain reliable transmit rate information from the QCA988x firmware is a crippling limitation that invalidates many of the most powerful software-based optimization techniques for improving spectral efficiency and link reliability. Advanced rate adaptation algorithms like Minstrel HT, while CONFIRMEDLY DEMONSTRATED on adjacent hardware (like the `ath9k` driver), are effectively INFERRED to be non-functional on the target QCA988x hardware. Similarly, fine-grained control over aggregation and retry policies is likely REFUTED as a practical avenue for

gain due to the combined effects of firmware offloading and missing telemetry. Any future work in this area must first address this telemetry gap, either through a patch to the `ath10k` driver or a modification to the underlying firmware itself.

Queuing Disciplines and Latency Reduction Strategies

While the prospects for enhancing the MAC and physical layers of the QCA988x hardware are constrained, there is significant potential for improving network performance through host-side software, specifically by employing advanced queuing disciplines. These techniques operate at the network layer on the host CPU, managing the flow of packets destined for the wireless interface. They do not change how the radio accesses the medium but can dramatically improve perceived latency, fairness, and overall responsiveness, which are critical metrics for a WISP environment. Key technologies in this domain include FQ-CoDel, CAKE, and the more recent Airtime Queue Limits (AQL).

Airtime Queue Limits (AQL) represent one of the most promising advancements for Wi-Fi fairness. AQL extends the concept of CoDel (Controlled-Delay) by incorporating airtime accounting [25](#). Instead of just limiting the delay a packet experiences in a queue, AQL limits the total amount of airtime a particular station can consume over a period. This directly combats the "polite people problem," where a few bandwidth-hungry clients can monopolize the channel and starve others, even if individual connections are healthy. The development of AQL was driven by the specific challenges of firmware-offloaded drivers like `ath10k`, acknowledging that MAC-layer control is often out of reach [25](#). The AQL feature was designed as a new kernel API and has been tested on Ubiquiti hardware, including UAP Mesh units and Archer C7v2 routers, demonstrating its relevance and potential portability to the target airMAX AC ecosystem [25](#). Implementing AQL on an OpenWrt-based FuturaMAX would be a high-priority objective, as it offers a direct path to improving fairness and stability in a congested PtMP sector.

Complementary to AQL are general-purpose traffic control queuing disciplines like FQ-CoDel (Flow Queue - Controlled Delay) and CAKE (Common Applications Kept Enhanced). FQ-CoDel works by sharding the main packet queue into many smaller queues, one for each flow. It then applies the CoDel active queue management algorithm to each flow's queue independently. This prevents a single TCP stream from dominating the queue and causing high latency for all other traffic, resulting in a more responsive network experience [95](#). Research on the `ath9k` driver showed how to build airtime

fairness on top of FQ-CoDel, proving the viability of this approach [95](#). More recent work aims to apply similar principles to `ath10k`, with AQL being a specific implementation tailored for that driver's needs [25](#). CAKE builds on these ideas, integrating traffic shaping, packet scheduling, and AQM into a single, easy-to-use framework designed to provide good performance across a wide range of network conditions. Both FQ-CoDel and CAKE are readily available in modern OpenWrt distributions and would be straightforward to configure on the AP side to improve latency and stability for all connected CPEs.

Another important aspect of queuing is pacing, which involves controlling the rate at which packets are handed off from the host CPU to the wireless driver for transmission. Proper pacing can smooth out bursts of traffic, preventing sudden congestion spikes at the radio. While less commonly configured than AQM, pacing is a key component of advanced traffic control systems like CAKE and is supported by the Linux kernel. On hardware that may become CPU-bound, especially at higher channel widths, ensuring that the host CPU is not overwhelmed by the task of feeding packets to the radio is crucial for maintaining stability [79](#). Although the QCA988x firmware handles MAC processing, the host still needs to assemble and submit frames, and excessive CPU load can become a bottleneck [79](#).

In summary, while direct manipulation of the radio's MAC and PHY layers is severely limited, the host-side software stack offers a fertile ground for optimization. Techniques focused on queuing discipline and traffic management are independent of the radio's firmware and can be implemented entirely within an OpenWrt environment. Among these, Airtime Queue Limits (AQL) stand out as the most targeted and promising solution for addressing the specific fairness and stability challenges of a PtMP WISP sector. Its development was motivated by the very constraints of firmware-offloaded drivers, making it a prime candidate for successful porting and deployment on the target Ubiquiti hardware.

Synthesis of Findings and Path Forward for FuturaMAX

This exhaustive investigation reveals that while the legacy Ubiquiti airMAX AC hardware possesses untapped potential for software-driven improvement, the path to realizing material gains is narrow and heavily constrained by architectural and proprietary factors. The analysis confirms that significant enhancements in capacity, spectral efficiency, and stability are achievable, but primarily through host-side queuing disciplines rather than invasive modifications to the radio's MAC layer. The most promising avenues are blocked

by fundamental limitations in firmware offloading and telemetry, which render many advanced Wi-Fi optimization techniques non-functional.

The strongest positive findings center on host-based traffic management. **Airtime Queue Limits (AQL)** has been specifically developed for ath10k-based hardware to combat the polite people problem inherent in PtMP networks [25](#). Its design acknowledges the realities of firmware offloading and its testing on Ubiquiti devices makes it a prime candidate for immediate implementation in a FuturaMAX project [25](#). Similarly, general-purpose queuing disciplines like **FQ-CoDel** and **CAKE** are proven technologies for improving latency and fairness and are readily available in OpenWrt [25](#) [95](#). These techniques do not require deep knowledge of the radio's inner workings and offer a direct route to improving the end-user experience. Finally, **stabilizing the underlying OpenWrt operating system** is a foundational step; many of the erratic behaviors reported by users on stock ai rOS may be symptoms of underlying OS instability that a custom, patched OpenWrt build could resolve.

Conversely, the investigation uncovered 20 significant blockers and negative findings that must be acknowledged. The single most critical blocker is the **lack of reliable transmit rate telemetry** from the QCA988x firmware, which cripples any adaptive rate control algorithm like Minstrel HT [40](#). This finding is CONFIRMED on the target hardware and effectively REFUTES the utility of advanced rate adaptation. Other major blockers include the **opaque, black-box nature of Ubiquiti's proprietary airMAX TDMA scheduler**, which prevents any form of external augmentation or control [23](#); **inconsistent telemetry in stock firmware**, which undermines any network-wide optimization strategy [24](#) [68](#); and **abandoned or unstable experimental implementations** in the Linux kernel that highlight the difficulty of MAC-layer modifications on this hardware [26](#). Fine-grained control over **aggregation and retry policies** is also likely REFUTED due to firmware offloading and missing controls.

The following table summarizes key findings and their portability to the target airMAX AC hardware.

Technique / Finding	Source(s)	Baseline	Modification Tested	Measured Outcome / Status	Portability to QCA988x
Airtime Queue Limits (AQL)	Kernel Patches 25	Standard ath10k with fq_codel	AQL kernel API + Userspace control	Improves fairness, reduces latency on ath10k and mt76 drivers. Tested on Ubiquiti UAP Mesh.	CONFIRMED: Designed for ath10k, high priority for porting to OpenWrt.
Minstrel HT Rate Adaptation	ath9k patches 95	Standard ath9k driver	Minstrel HT algorithm	Increased throughput and improved fairness on ath9k.	REFUTED: Effectiveness is CONFIRMEDLY BLOCKED by lack of transmit rate telemetry in ath10k 40 .
Lack of Tx Rate Telemetry	ath10k Driver Code 40	Stock ath10k driver	N/A (Fundamental limitation)	tx_status.rate_idx is unreliable; correct rate requires parsing debugfs files.	CONFIRMED: A core limitation of the target hardware/firmware stack.
Proprietary airMAX TDMA Scheduler	Ubiquiti Docs 23 117 , Patents 84	None (Closed system)	N/A (Opaque component)	Eliminates hidden-node collisions and maximizes airtime efficiency.	BLOCKED: No external control or telemetry interfaces documented.
Stock airOS Instability	Ubiquiti Community Forums 50 54 68	Various airOS versions	N/A (System behavior)	Reports of dropped connections, CCQ drops, CLI hangs, and management incompatibility.	DEMONSTRATED: Observed on various airMAX AC devices.
ath10k TXQ Scheduling API Work	Linux Wireless ML 26	Standard ath10k driver	Experimental TXQ scheduling API patches	Patchset proposed but development hit walls due to firmware offloading complexities.	ABANDONED/UNSTABLE: Conceptually possible but practically difficult due to architecture.
Alternative ath10k Firmwares	OpenWrt Wiki 38 39	Official QCA firmware	Candela Technologies firmware	Enables IBSS mode and other features. Does not fix telemetry gap.	INFERRED: May improve stability/features but does not solve core limitation.
Spectral Scan for Interference	Candela Tech Docs 66 , FCC Docs 48	Standard 802.11 operation	Utilizing spectral scan data	Provides interference data. Real-time mitigation feasibility is UNKNOWN.	UNKNOWN: Hardware capability exists, but software to act on it is not described.

Based on this synthesis, the following ranked list of techniques is most likely to yield positive results:

1. **Implement Airtime Queue Limits (AQL):** Highest priority. Directly addresses fairness and latency, designed for ath10k.
2. **Configure Host-Side Traffic Shaping (FQ-CoDel/CAKE):** High priority. Improves responsiveness and stability on the host side.
3. **Stabilize the Base OS (OpenWrt):** Foundational. Resolves many of the instability issues reported in stock airOS.
4. **Investigate External TDMA Augmentation:** Medium priority, high risk. Requires reverse-engineering airOS control interfaces, which is currently unknown.
5. **Develop Per-CPE Historical Models:** Low priority. Dependent on resolving the telemetry gap to collect sufficient, clean data.

The following ten questions can only be answered through physical experimentation:

1. **Can an external controller successfully interface with and influence the airOS TDMA scheduler?** (Requires probing REST API [97](#) or other undocumented interfaces).
2. **What is the true relationship between reported RSSI/SNR and actual achievable TCP throughput?** (Requires controlled iperf tests).
3. **Is the "unstable rate mask" issue a driver bug or a firmware limitation?** (Requires testing different ath10k firmwares on a single device).
4. **How does AQL perform in a live, congested PtMP sector compared to airOS's default behavior?** (The ultimate validation test).
5. **What is the maximum sustainable CPU load of the target hardware before becoming a bottleneck?** (Crucial for running an external controller [79](#)).
6. **Does enabling/disabling specific firmware features (e.g., AC mode) via downgrade/revert affect the core scheduler's performance?** (Tests firmware regressions [50](#)).
7. **Can fine-tuning RTS/CTS thresholds improve performance in a specific hidden-node topology?** (Requires empirical testing of different values).
8. **Is there a difference in performance between GPS-synchronized and non-GPS-enabled variants under identical RF conditions?** (Compares LAP-GPS vs. Loco 5AC behavior).
9. **How does the system behave under mixed client conditions (e.g., old 802.11n clients alongside AC clients)?** (Tests backward compatibility and coexistence).
10. **What is the impact of different A-MPDU aggregation settings on both throughput and error resilience?** (Requires testing variable A-MPDU sizes).

In conclusion, the FuturaMAX project should pivot its focus away from attempting to reinvent the MAC layer and toward leveraging the power of host-side queuing and traffic management. The most credible and reproducible path to material improvement lies in the careful implementation and tuning of AQL and related technologies within a stable OpenWrt environment.

Reference

1. airMagic spectral efficiency <https://community.ui.com/questions/airMagic-spectral-efficiency/ce4847c6-d8c0-4da5-8309-a97b5f823907>

2. Publications - SpectrumX <https://www.spectrumx.org/publications/>
3. 802.11p Spectral Emission Mask Testing <https://www.mathworks.com/help/wlan/ug/802-11p-spectral-emission-mask-testing.html>
4. SPECTRAL EFFICIENCY AND DETECTION EFFICIENCY OF ... <https://repository.arizona.edu/items/78879a71-5fed-4497-a01e-4e73c246a5dc>
5. Spectral Efficiency Improvement of Generalized Frequency ... <https://etasr.com/index.php/ETASR/article/view/16649>
6. 5G Researchers Set New World Record For Spectrum ... <https://spectrum.ieee.org/5g-researchers-achieve-new-spectrum-efficiency-record>
7. AGU26 <https://agu.confex.com/agu/agu26/meetingapp.cgi/Paper/2062999>
8. Spectral Efficiency Analysis for IRS-Assisted MISO ... <https://www.mdpi.com/2227-7390/11/14/3181>
9. Study of Spectral Efficiency for LTE Network https://www.asrjetsjournal.org/American_Scientific_Journal/article/download/2662/1065/8251
10. Understanding open source wifi drivers (and other wifi related ... https://www.reddit.com/r/openwrt/comments/171pw7d/understanding_open_source_wifi_drivers_and_other/
11. airMAX AC 8.7.18 - Ubiquiti Community <https://community.ui.com/releases/airMAX-AC-8-7-18/2c16f204-a75b-4691-a12e-4e3365aeb939>
12. Avoid latest Ubiquiti firmware update for custom settings - Facebook <https://www.facebook.com/groups/ubiquitisggroup/posts/474809694403988/>
13. RTCM Client - "AllStarLink Wiki" - DVSwitch http://dvswitch.org/files/AllStarLink/Voter/RTCM%20Client%20-%20_AllStarLink%20Wiki_.html
14. Unifi network application 9.2.x updates with new dhcp manager ... <https://www.facebook.com/groups/uitaiwan.group/posts/4189229174629255/>
15. [PDF] PROYECTO DE INSTALACIÓN RADIOELÉCTRICA PARA UNA ... <https://transparencia.tocinalosrosales.es/export/sites/tocina/es/transparencia/.galleries/IND-26-/INSTALACION-RADIOELECTRICA-PARA-RED-DE-TELECOMUNICACIONES-INLAMBRICA/001.-P20301186-GA183041TEC-RAD-DISOPL-WIFOPL-JEYCA-SEVILLATOCINAv2020.pdf>
16. Ubiquity products just arrived!!! Available in Dolores San Fernando ... <https://www.facebook.com/pclogic.ph/posts/ubiquity-products-just-arrivedavailable-in-dolores-san-fernando-and-balibago-ang/1156662601158003/>
17. Network - Phnom Penh - Khmer Plus Computer <https://www.kpcccomputer.com/network>
18. Nano Loco 5AC Internacional 180 USD - Facebook <https://www.facebook.com/groups/571699864265496/posts/1793911895377614/>

19. Unifi pro xg8 and u7 pro xgs connection issues? - Facebook <https://www.facebook.com/groups/1530987921465232/posts/1561795721717785/>
20. UBIQUITI LITEAP AC LAP-120 5GHz AIRMAX AC SECTOR 2x2 ... <https://www.facebook.com/DFE168/posts/ubiquiti-liteap-ac-lap-120-5ghz-airmax-ac-sector-2x2-mimo-ap-lap-120php-495000-d/3562892050474048/>
21. Software Downloads - Ubiquiti <http://downloads.ubnt.com/airmax-ac>
22. Software Downloads - Ubiquiti <https://www.ubnt.com/software/>
23. [PDF] What Is AirMax? - Ubiquiti https://dl.ubnt.com/AirMax_ppt.pdf
24. Release Notes - Ubiquiti <https://dl.ubnt.com/firmwares/XC-fw/v8.7.11/changelog.txt>
25. AQL and the ath10k is *lovely* - For Developers <https://forum.openwrt.org/t/aql-and-the-ath10k-is-lovely/59002>
26. Lets make wifi fast again! - public-inbox listing <https://lists.bufferbloat.net/make-wifi-fast/?t=20180427103219>
27. [OpenWrt-Devel] [PATCH 0/3 v2] mac80211 <https://lists.openwrt.org/pipermail/openwrt-devel/2015-June/007876.html>
28. airMAX - Frequently Asked Questions (FAQs) – UISP Help Center <https://help.uisp.com/hc/en-us/articles/22590834519959-airMAX-Frequently-Asked-Questions-FAQs>
29. [PDF] DATASHEET - Estec <https://www.estec.cl/media/catalog/product/file/P07765.pdf>
30. airMAX Wireless Best Practices | HostiFi Help Center <https://support.hostifi.com/en/articles/6300438-airmax-wireless-best-practices>
31. Doing my first Wireless Site Survey - Ubiquiti Community <https://community.ui.com/questions/Doing-my-first-Wireless-Site-Survey/3f10e647-3de6-4aed-b646-102e0ab32b32>
32. Wireless Site Surveys Explained: Tools, Methods, and Best Practices <https://www.youtube.com/watch?v=0y2Mw-3ajbY>
33. Analyzing Wi-Fi Site Survey Visualizations - NetAlly <https://www.netally.com/tech-tips/analyzing-wifi-site-survey-visualizations-the-easy-way/>
34. How to Conduct a Wireless Site Survey? - ExterNetworks Inc. <https://www.extnoc.com/learn/networking/conduct-wireless-site-survey/>
35. Your wireless network may perform flawlessly in the lab, but real ... <https://www.facebook.com/vvdntech/posts/your-wireless-network-may-perform-flawlessly-in-the-lab-but-real-world-deploymen/1426309339517680/>
36. Wireless site survey best practices - wifi-bond <https://wifibond.com/2017/08/17/wireless-site-survey-best-practices/>

37. Top 5 Wireless Site Survey Best Practices | FieldEngineer <https://www.fieldengineer.com/blogs/site-survey-best-practices>
38. ath10k-firmware-qca9888-ct - [OpenWrt Wiki] package <https://openwrt.org/packages/pkgdata/ath10k-firmware-qca9888-ct>
39. ath10k-firmware-qca988x-ct - [OpenWrt Wiki] package <https://openwrt.org/packages/pkgdata/ath10k-firmware-qca988x-ct>
40. About ath10k — Linux Wireless documentation <https://wireless.docs.kernel.org/en/latest/en/users/drivers/ath10k.html>
41. en/users/Drivers/ath10k - Linux Wireless - SIP Solutions https://linuxwireless.sipsolutions.net/en/users/Drivers/ath10k/_v32.html
42. Mitigating Co-Channel Interference for Maximum Spectral Efficiency <https://inis.iaea.org/records/g8mdz-rev58>
43. Co-channel interference cancellation in reuse-1 deployments of WiMAX ... https://www.researchgate.net/publication/254037597_Co-channel_interference_cancellation_in_reuse-1_deployments_of_WiMAX_system
44. How Much does Massive MIMO Improve the Spectral Efficiency? <https://ma-mimo.ellintech.se/2016/10/18/how-much-does-massive-mimo-improve-spectral-efficiency/>
45. Energy and Spectral Efficiency Analysis for UAV-to-UAV ... - MDPI <https://www.mdpi.com/2624-6511/8/2/54>
46. [PDF] Comparison of Spectral and Energy Efficiency Metrics using ... https://tma.ifip.org/2018/wp-content/uploads/sites/3/2018/06/tma2018_paper5.pdf
47. [PDF] Challenges and Considerations in Defining Spectrum Efficiency <https://rysavy.com/wp-content/uploads/2017/08/2014-03-ieee-defining-spectrum-efficiency.pdf>
48. Research Activities - Wireless Communications & Signal Processing <http://wcsp.eng.usf.edu/research.html>
49. [PDF] Contributions to Analysis and Mitigation of Cochannel ... <https://theses.eurasip.org/wp-content/uploads/cierny-michal-contributions-to-analysis-and-mitigation-of-cochannel-interference-in-cellular-wireless-networks.pdf>
50. airMAX AC Firmware - Ubiquiti Community <https://community.ui.com/releases/2f500636-33f7-4134-aad2-df3ddcf1a9e6?replyId=6e388b7b-d2a6-4677-b084-760e9aacda9b>
51. airMAX AC Firmware - Ubiquiti Community <https://community.ui.com/releases/airMAX-AC-Firmware-v8-7-0/2f500636-33f7-4134-aad2-df3ddcf1a9e6>
52. best airmax fw version? - Ubiquiti Community <https://community.ui.com/questions/best-airmax-fw-version/09c5533e-b49b-4367-913c-391cb8076ebc>

53. airMAX AC - UI Community - Ubiquiti <https://community.ui.com/releases/airMAX-AC-8-7-4/6b38ecfe-8486-40c0-9be5-5fb168c7a010?page=2>
54. airMAX AC - Ubiquiti Community <https://community.ui.com/releases/airMAX-AC-8-7-19/f60c28c6-d470-4ba3-9981-192bb6aca713>
55. airMAX AC - Ubiquiti Community <https://community.ui.com/releases/airMAX-AC-8-7-4/6b38ecfe-8486-40c0-9be5-5fb168c7a010>
56. airMAX AC - Ubiquiti Community <https://community.ui.com/releases/airMAX-AC/804eb7e2-1790-4143-8e98-2e8c0e29463e>
57. airMAX AC Firmware - Ubiquiti Community <https://community.ui.com/releases/airMAX-AC-Firmware-v8-7-0/2f500636-33f7-4134-aad2-df3ddcf1a9e6?page=8>
58. [Airmax AC/Bug report] Unable for client side of a PTP link to ... <https://community.ui.com/questions/Airmax-AC-Bug-report-Unable-for-client-side-of-a-PTP-link-to-connect-back-after-a-frequency-change-/f37b0212-993a-4dd2-bf39-479847d0bcf8>
59. Google Scholar https://scholar.google.com/_
60. Atoosa Dalili Shoei - Google Scholar <https://scholar.google.com/citations?user=ZkZoY3EAAAAJ&hl=en>
61. Rui ZOU - Google Scholar <https://scholar.google.com/citations?user=upjCKFYAAAAJ&hl=en>
62. Abdeldime Mohamed Salih Abdelgader - Google Scholar <https://scholar.google.com/citations?user=CjprMXAAAAAJ&hl=en>
63. Prasad Anjani - Google Scholar <https://scholar.google.com/citations?user=EUQAZGAAAAAJ&hl=en>
64. Liangping Ma - Google Scholar <https://scholar.google.com/citations?user=tWSGUdoAAAAJ&hl=en>
65. suneel kumar - Google Scholar <https://scholar.google.com/citations?user=gKD8qkQAAAAJ&hl=en>
66. Ath10k Candela Technologies CT 10.1 Firmware <https://www.candelatech.com/ath10k-10.1.php>
67. 2: RTS and CTS address the Hidden Node Problem - ResearchGate https://www.researchgate.net/figure/RTS-and-CTS-address-the-Hidden-Node-Problem_fig2_2533138
68. Slow airMAX AC speeds on PtMP to PtP link - Ubiquiti Community <https://community.ui.com/questions/Slow-airMAX-AC-speeds-on-PtMP-to-PtP-link/b6a17936-809f-47e0-acf6-215c825e3a41>
69. AirMax signal issues - Ubiquiti Community <https://community.ui.com/questions/AirMax-signal-issues/59e55d65-e88d-41d3-938d-b9a3450ce362?page=1>

70. Ubiquiti AirMAX ac Sectors | NetWifiWorks.com <https://www.netwifiworks.com/airMAX-ac-Sectors.asp>
71. airMAX AC Firmware 8.6.1 - Ubiquiti Community <https://community.ui.com/releases/airMAX-AC-Firmware-8-6-1/9024171e-8043-4260-a8b5-1a75490bbd3e>
72. Most stable latest AirMax firmware? - UI Community - Ubiquiti <https://community.ui.com/questions/Most-stable-latest-AirMax-firmware/cc9c8581-99b8-493f-ba73-fdb6d98c3625>
73. airMAX AC Firmware - UI Community - Ubiquiti <https://community.ui.com/releases/airMAX-AC-Firmware-v8-7-1/8a609e25-ab5d-4640-9ebc-191c8e280087?page=4>
74. definitions of efficiency in spectrum use october 1, 2008 https://www.ntia.gov/sites/default/files/publications/spectral_efficiency_final_0.pdf
75. Spectral efficiency https://en.wikipedia.org/wiki/Spectral_efficiency
76. Evaluation of Spectral Efficiency, System Capacity And ... <https://thesai.org/Downloads/Volume3No6/Paper%205-EVALUATION%20OF%20SPECTRAL%20EFFICIENCY,%20SYSTEM%20CAPACITY%20AND%20INTERFERENCE%20EFFECTS%20ON%20CDMA%20COMMUNICATION%20SYSTEM.pdf>
77. Spectral Efficiency Improvement of Generalized Frequency ... <https://etasr.com/index.php/ETASR/article/download/16649/6757>
78. Combining spectral efficiency with user experience <https://scispace.com/pdf/user-spectral-efficiency-combining-spectral-efficiency-with-18nqeo3pp.pdf>
79. AirMAX AC Devices can not reach the speed they are sold at <https://community.ui.com/questions/AirMAX-AC-Devices-can-not-reach-the-speed-they-are-sold-at/faa52594-c6a0-4375-95e8-33bd6b417f89>
80. A Survey on Multi-AP Coordination Approaches over Emerging ... <https://arxiv.org/html/2306.04164v2>
81. Spectral efficiency - YouTube <https://www.youtube.com/watch?v=8GIbrW4DSWY>
82. The Importance of Spectral Efficiency - Tarana Wireless <https://taranawireless.com/the-importance-of-spectral-efficiency/>
83. What is Spectral Efficiency? - AccelerComm <https://www.accelercomm.com/spectral-efficiency>
84. Patents Assigned to Ubiquiti Inc. <https://patents.justia.com/assignee/ubiquiti-inc>
85. Spectral Efficiency Improvement Using Bi-Deep Learning Model for ... <https://www.mdpi.com/1424-8220/23/18/7793>
86. The Effect of Pilot Reuse Factor on Massive MIMO Spectral Efficiency <https://etasr.com/index.php/ETASR/article/view/5605>

87. Spectral Efficiency Improvement With 5G Technologies https://www.researchgate.net/publication/317420269_Spectral_Efficiency_Improvement_With_5G_Technologies_Results_From_Field_Tests
88. Coordinated TDMA MAC Scheme Design and Performance Evaluation ... https://link.springer.com/chapter/10.1007/978-3-030-69514-9_24
89. Performance Analysis of IEEE 802.11bn with Coordinated TDMA ... <https://arxiv.org/html/2508.18755v2>
90. Fully Distributed TDMA Scheduling Protocols <https://www.emergentmind.com/topics/fully-distributed-tdma-scheduling-protocols>
91. A Hybrid Link – TDMA MAC Protocol for Conventional and Radio ... <https://onlinelibrary.wiley.com/doi/10.1155/2020/9340272>
92. A Survey of TDMA Scheduling Schemes in Wireless Multihop ... <https://dl.acm.org/doi/10.1145/2677955>
93. Learning to allocate: a delay and temperature-aware slot ... <https://www.semanticscholar.org/paper/Learning-to-allocate%3A-a-delay-and-temperature-aware-Mystica-Martin/2069a9b70b06ee6e5415832852dec8a474cd6a78>
94. [PDF] AP-initiated Multi-User Transmissions in IEEE 802.11ax WLANs - arXiv <https://arxiv.org/pdf/1702.05397>
95. FS#368 - issues with airtime fairness on the ath9k #5455 - GitHub <https://github.com/openwrt/openwrt/issues/5455>
96. New airMAX AC Firmware v8.7.4 - Ubiquiti Community <https://community.ui.com/questions/New-airMAX-AC-Firmware-v8-7-4/01fd1325-be5e-4288-9135-eb0b9bca7f75>
97. REST client for airmac ac with python : r/Ubiquiti - Reddit https://www.reddit.com/r/Ubiquiti/comments/1l4gibr/rest_client_for_airmax_ac_with_python/
98. Spectral Efficiency Calculator - TELCOMA Global <https://www.telcomaglobal.com/p/spectral-efficiency-calculator>
99. [PDF] Spectral Efficiency Considerations for 6G - arXiv <https://arxiv.org/pdf/2508.09117>
100. Spectral Efficiency | Bits/Second/Hz | 5G-4G <https://www.techplayon.com/spectral-efficiency-5g-nr-and-4g-lte/>
101. [PDF] Ubiquiti® NBE-5AC-GEN2 | airMAX® ac CPE with Dedicated ... https://www.unixcctv.com/product/ubiquiti-nbe-5ac-gen2-airmax-ac-cpe-with-dedicated-management-radio/?print-products=pdf_g
102. Real-world Litebeam 5AC speeds lower than advertised in AirOS ... https://www.reddit.com/r/Ubiquiti/comments/1cuyk5f/realworld_litebeam_5ac_speeds_lower_than/

103. How America's Wireless Industry Maximizes Its Spectrum https://api.ctia.org/wp-content/uploads/2019/07/Spectrum_Efficiency.pdf
104. airMAX AC 8.7.17 - UI Community - Ubiquiti <https://community.ui.com/releases/airMAX-AC-8-7-17/83aa5a01-d883-4d82-846a-71c33ba32f68>
105. For the Ubiquiti airMAX fans among us, airOS 8.7.12 has ... - Facebook <https://www.facebook.com/hostifinet/posts/for-the-ubiquiti-airmax-fans-among-us-airos-8712-has-just-been-released-marking-/913528070775908/>
106. airMAX AC | Ubiquiti Community <https://community.ui.com/releases/airMAX-AC-8-7-15/8683e08d-fd52-4361-a801-4cba0ff38ef9>
107. Ubiquiti LiteBeam 5AC · Issue #149884 · home-assistant/core - GitHub <https://github.com/home-assistant/core/issues/149884>
108. Performance Analysis of IEEE 802.11bn with Coordinated ... <https://arxiv.org/pdf/2508.18755>
109. Tokar_LO_Krasnozheniuk_YO.doc (463.5 KB) <https://openarchive.nure.ua/bitstreams/edbe4773-d019-4aae-b9c9-acca09c75a6c/download>
110. [Feature Request] Airtime Fairness for wireless package <https://forum.mikrotik.com/t/feature-request-airtime-fairness-for-wireless-package/162081>
111. arXiv:2302.10676v1 [cs.NI] 21 Feb 2023 <https://arxiv.org/pdf/2302.10676>
112. TDMA Filter and Frame Duration on a Rocket AC Prism with M ... <https://community.ui.com/questions/3e9165f6-979a-4050-a0d0-42fcc31c8101>
113. Reading system logs - Ubiquiti Community <https://community.ui.com/questions/Reading-system-logs/6a55a228-77bc-406c-a969-ebe0a3e726fb>
114. CodeFetch/Ubiquiti-UBNT-airOS: GPL source code of ... - GitHub <https://github.com/CodeFetch/Ubiquiti-UBNT-airOS>
115. Ubiquiti AirOS · Issue #151145 · home-assistant/core - GitHub <https://github.com/home-assistant/core/issues/151145>
116. misc-scripts/ubiquiti-mac-acl/wireless.sql at master - GitHub <https://github.com/jantman/misc-scripts/blob/master/ubiquiti-mac-acl/wireless.sql>
117. [PDF] airMAX TDMA Technology Datasheet - Ubiquiti https://dl.ubnt.com/datasheets/airmax/UBNT_DS_airMAX_TDMA.pdf